

Assignment 3 – SQL QUERIES

Name: Aryan Ligade

Prn-124b1f155

class/div-Sy(A4)

Case Study: Employee Performance Appraisal System

1. Aim

To design and execute SQL queries using Joins, Subqueries, Views, GROUP BY, and HAVING clauses for efficient data retrieval and analysis in a relational database system.

2. Problem Statement (Case Study)

- a. Write at least 10 SQL queries for suitable database application using SQL DML statements using joins and subqueries.
- b. Write queries using group by and having clause.

3. Database Design

1. Passenger Entity

This entity stores details of passengers.

- **passenger_id** → Integer, Primary Key
- **passenger_name** → String (cannot be NULL)
- **passport_no** → String (must be unique)
- **country** → String

Each passenger is uniquely identified by **passenger_id**.

◇ 2. Flight Entity

This entity stores flight details.

- `flight_id` → Integer, Primary Key
- `flight_number` → String (unique)
- `source` → String (departure location)
- `destination` → String (arrival location)
- `departure_time` → DateTime

Each flight is uniquely identified by `flight_id`.

◇ 3. Booking Entity

This entity connects Passenger and Flight (relationship table).

- `booking_id` → Integer, Primary Key
- `passenger_id` → Foreign Key referencing Passenger
- `flight_id` → Foreign Key referencing Flight
- `booking_date` → Date
- `seat_no` → String

This table resolves many-to-many relationship between Passenger and Flight.

◇ 4. Staff Entity

This entity stores staff assigned to flights.

- `staff_id` → Integer, Primary Key
- `staff_name` → String
- `role` → String (Pilot, Crew, etc.)

- flight_id → Foreign Key referencing Flight

One flight can have multiple staff members.

4. Database Schema

```
mysql> USE airline_db;
Database changed
mysql> CREATE TABLE Passenger (
->     passenger_id INT PRIMARY KEY,
->     passenger_name VARCHAR(100),
->     passport_no VARCHAR(20),
->     country VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Flight (
->     flight_id INT PRIMARY KEY,
->     flight_number VARCHAR(20),
->     source VARCHAR(50),
->     destination VARCHAR(50),
->     departure_time DATETIME
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Booking (
->     booking_id INT PRIMARY KEY,
->     passenger_id INT,
->     flight_id INT,
->     booking_date DATE,
->     seat_no VARCHAR(10),
->     FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id),
->     FOREIGN KEY (flight_id) REFERENCES Flight(flight_id)
-> );
Query OK, 0 rows affected (0.04 sec)
```

5. SQL Inputs and Outputs

1. INNER JOIN

a) Display Passenger Name with Flight Number

```
mysql> SELECT p.passenger_name, f.flight_number
-> FROM Passenger p
-> INNER JOIN Booking b ON p.passenger_id = b.passenger_id
-> INNER JOIN Flight f ON b.flight_id = f.flight_id;
```

passenger_name	flight_number
Rahul	AI101
Aryan	AI101

2 rows in set (0.00 sec)

2: Show Booking Details with Flight Route-

```
mysql> SELECT
-> p.passenger_name,
-> f.source,
-> f.destination,
-> b.seat_no,
-> b.booking_date
-> FROM Booking b
-> JOIN Passenger p ON b.passenger_id = p.passenger_id
-> JOIN Flight f ON b.flight_id = f.flight_id;
```

passenger_name	source	destination	seat_no	booking_date
Rahul	Delhi	Mumbai	B2	2026-04-15
Aryan	Delhi	Mumbai	A1	2026-04-15

2 rows in set (0.00 sec)

3 :LEFT JOIN-

a) Show All Passengers (Even If No Bookings)

```
mysql> SELECT p.passenger_name, b.booking_id
      -> FROM Passenger p
      -> LEFT JOIN Booking b ON p.passenger_id = b.passenger_id;
```

passenger_name	booking_id
Aryan	1
Rahul	2
John	NULL

3 rows in set (0.00 sec)

4: RIGHT JOIN-

Show All Flights (Even If No Passenger Booked)

```
mysql> SELECT f.flight_number, b.booking_id
      -> FROM Booking b
      -> RIGHT JOIN Flight f ON b.flight_id = f.flight_id;
```

flight_number	booking_id
AI101	1
AI101	2
AI202	NULL

3 rows in set (0.00 sec)

5: FULL JOIN-

Show All Passengers and All Bookings

```
mysql> SELECT p.passenger_name, b.booking_id
-> FROM Passenger p
-> LEFT JOIN Booking b ON p.passenger_id = b.passenger_id
->
-> UNION
->
-> SELECT p.passenger_name, b.booking_id
-> FROM Passenger p
-> RIGHT JOIN Booking b ON p.passenger_id = b.passenger_id;
```

passenger_name	booking_id
Aryan	1
Rahul	2
John	NULL

3 rows in set (0.00 sec)

6: CROSS JOIN-

All Passenger-Flight Combinations

```
mysql> SELECT p.passenger_name, f.flight_number  
-> FROM Passenger p  
-> CROSS JOIN Flight f;
```

passenger_name	flight_number
Aryan	AI202
Aryan	AI101
Rahul	AI202
Rahul	AI101
John	AI202
John	AI101

6 rows in set (0.00 sec)

7: SELF JOIN-

Find Flights Having Same Source

```
mysql> SELECT F1.flight_number, F2.flight_number, F1.source  
-> FROM Flight F1  
-> JOIN Flight F2  
-> ON F1.source = F2.source  
-> AND F1.flight_id <> F2.flight_id;  
Empty set (0.00 sec)
```

8: Passengers with Maximum Booking Amount-

```
mysql> SELECT p.passenger_name, COUNT(b.booking_id) AS total_bookings
-> FROM Passenger p
-> JOIN Booking b ON p.passenger_id = b.passenger_id
-> GROUP BY p.passenger_id
-> HAVING COUNT(b.booking_id) = (
->     SELECT MAX(cnt)
->     FROM (
->         SELECT COUNT(*) AS cnt
->         FROM Booking
->         GROUP BY passenger_id
->     ) AS temp
-> );
```

passenger_name	total_bookings
Aryan	1
Rahul	1

2 rows in set (0.01 sec)

9: Flights with Above Average Bookings-

```
mysql> SELECT F.flight_number
-> FROM Flight F
-> JOIN Booking B ON F.flight_id = B.flight_id
-> GROUP BY F.flight_id
-> HAVING COUNT(*) > (
->     SELECT AVG(total_count)
->     FROM (
->         SELECT COUNT(*) AS total_count
->         FROM Booking
->         GROUP BY flight_id
->     ) AS temp
-> );
```

Empty set (0.01 sec)

10: Passengers Who Booked More Than One Flight-

```
mysql> SELECT flight_number
-> FROM Flight
-> WHERE flight_id NOT IN (
->     SELECT flight_id FROM Booking
-> );
```

Empty set (0.00 sec)

B) **JOIN with GROUP BY**-

```
mysql> SELECT f.flight_number, COUNT(b.booking_id) AS total_passengers
-> FROM Flight f
-> LEFT JOIN Booking b ON f.flight_id = b.flight_id
-> GROUP BY f.flight_number;
+-----+-----+
| flight_number | total_passengers |
+-----+-----+
| AI101         | 2                |
| AI202         | 0                |
+-----+-----+
2 rows in set (0.00 sec)
```

JOIN with HAVING-

```
mysql> SELECT f.flight_number, COUNT(b.booking_id) AS total_passengers
-> FROM Flight f
-> JOIN Booking b ON f.flight_id = b.flight_id
-> GROUP BY f.flight_number
-> HAVING COUNT(b.booking_id) > 1;
+-----+-----+
| flight_number | total_passengers |
+-----+-----+
| AI101         | 2                |
+-----+-----+
1 row in set (0.00 sec)
```

Insert records-

```
mysql> INSERT INTO Passenger VALUES
    -> (1, 'Aryan', 'P12345', 'India'),
    -> (2, 'Rahul', 'P67890', 'India'),
    -> (3, 'John', 'P54321', 'USA');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Flight VALUES
    -> (101, 'AI101', 'Delhi', 'Mumbai', '2026-04-20 10:00:00'),
    -> (102, 'AI202', 'Pune', 'Dubai', '2026-04-21 12:00:00');
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Booking VALUES
    -> (1, 1, 101, '2026-04-15', 'A1'),
    -> (2, 2, 101, '2026-04-15', 'B2');
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

6. Output-

```
mysql> SELECT * FROM Passenger;
+-----+-----+-----+-----+
| passenger_id | passenger_name | passport_no | country |
+-----+-----+-----+-----+
| 1 | Aryan | P12345 | India |
| 2 | Rahul | P67890 | India |
| 3 | John | P54321 | USA |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Flight;
+-----+-----+-----+-----+-----+
| flight_id | flight_number | source | destination | departure_time |
+-----+-----+-----+-----+-----+
| 101 | AI101 | Delhi | Mumbai | 2026-04-20 10:00:00 |
| 102 | AI202 | Pune | Dubai | 2026-04-21 12:00:00 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Booking;
+-----+-----+-----+-----+-----+
| booking_id | passenger_id | flight_id | booking_date | seat_no |
+-----+-----+-----+-----+-----+
| 1 | 1 | 101 | 2026-04-15 | A1 |
| 2 | 2 | 101 | 2026-04-15 | B2 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

7. Conclusion

The study and implementation of SQL queries using **Joins, Subqueries, Views, GROUP BY, and HAVING clauses** play a crucial role in efficient database management and data

retrieval. Through this practical, we understood how relational databases organize data into multiple tables and how these tables can be logically connected using keys.

By using different types of **Joins** such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and others, we were able to combine data from multiple tables and extract meaningful information. Joins helped in understanding relationships between entities like Passenger, Flight, and Booking, making it easier to retrieve combined data in a single query.

The use of **Subqueries** enhanced the flexibility of SQL by allowing queries within queries. This helped in solving complex problems such as finding maximum values, filtering records based on aggregated results, and performing conditional checks dynamically. Correlated subqueries further improved query precision by executing row-wise operations.

The implementation of **Views** provided an abstraction layer over complex queries. Views simplified data access, improved security by restricting direct table access, and allowed reuse of frequently used queries without rewriting them.

Additionally, the use of **GROUP BY and HAVING clauses** enabled data aggregation and filtering of grouped results. These clauses were useful for generating summaries, such as counting passengers per flight or identifying flights with a high number of bookings.

Overall, this practical demonstrated how SQL DML operations combined with advanced querying techniques help in managing and analyzing large datasets efficiently. It also highlighted the importance of proper database design, normalization, and the use of keys to maintain data integrity and consistency.